MalVision: Using Deep Convolutional Neural Networks and Image Visualization

PROJECT REPORT

Submitted by

AGNIJ T DEV (TVE23CS014)

ANZER FAROOQ GANIE (TVE23CS041)

MALEEHA BEEFATHIMA (TVE23CS088)

NIYA P SHERRY (TVE23CS107)

to

The APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the award of the Degree

of

Bachelor of Technology

in

Computer Science and Engineering

Department of Computer Science and Engineering

College of Engineering Trivandrum

Kerala

April 2026

DECLARATION

We undersigned hereby declare that the project report **MalVision: Using Deep Convolutional Neural Networks and Image Visualization** , submitted for partial fulfillment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by us under supervision of **Asst Prof. Bincy P Mathew**. This submission represents our ideas in our own words and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Place: Thiruvananthapuram

Date: April 2026

AGNIJ T DEV

ANZER FAROOQ GANIE

MALEEHA BEEFATHIMA

NIYA P SHERRY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COLLEGE OF ENGINEERING TRIVANDRUM

CERTIFICATE

This is to certify that the report entitled **" MalVision: Using Deep Convolutional Neural Networks and Image Visualization ",** submitted by **AGNIJ T DEV** (TVE23CS014), **ANZER FAROOQ GANIE** (TVE23CS041), **MALEEHA BEEFATHIMA** (TVE23CS088), **NIYA P SHERRY** (TVE23CS107) to the **APJ Abdul Kalam Technological University** in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering is a bonafide record of the project work carried out by them under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Bincy P Mathew	**Ramcy R**	**Dr. Ajeesh Ramanujan**
Assistant Professor	Assistant Professor	Associate Professor
Department of CSE	Department of CSE	Department of CSE
(Guide)	(Coordinator)	(Head of Department)

ACKNOWLEDGEMENT

First and foremost, we would like to express our sincere gratitude and heartful indebtedness to our guide **Bincy P Mathew, Assistant Professor** , Department of Computer Science and Engineering for his/her valuable guidance and encouragement in pursuing this project.

We are also thankful to **Dr. Ajeesh Ramanujan**, Head of the Department for his help and support. We also extend our hearty gratitude to project co-ordinators, **Asst Prof.Ramcy R and Asst Prof. Swabna** College of Engineering Trivandrum for providing necessary facilities and their sincere co-operation.

Our sincere thanks is extended to all the teachers of the Department of Computer Science and Engineering and to all my friends for their help and support.

Above all, we thank God for the immense grace and blessings at all stages of the project.

AGNIJ T DEV

ANZER FAROOQ GANIE

MALEEHA BEEFATHIMA

NIYA P SHERRY

ABSTRACT

The rapid evolution of cyber threats, has rendered traditional signature-based detection and manual reverse-engineering increasingly ineffective. Conventional scanners are easily bypassed by zero-day attacks and polymorphic malware that constantly alters its code structure to hide known "fingerprints". Furthermore, manual dynamic analysis is time-consuming, requires risky isolated environments, and lacks the scalability to meet modern threat levels.

MalVision addresses these challenges by proposing a machine learning-based application that classifies malware through static image visualization. The core methodology involves converting raw binary executables, such as .exe or .dll files, into 8-bit grayscale images. These images are standardized to a 224×224 pixel resolution to ensure compatibility with a VGG16 Deep Convolutional Neural Network (CNN). By treating malware as an image, the system identifies malicious "textures" and structural patterns that remain consistent even when traditional text-based signatures are obfuscated or hidden.

The system architecture consists of four primary modules: File Conversion, Image Generation, CNN Classification, and Result Analysis. This pipeline allows for safe analysis as the malicious code is never executed; instead, it is processed in milliseconds to provide a final malware family label and a statistical confidence score.

Currently, the MalVision prototype has successfully utilized transfer learning to achieve over 97.28% validation accuracy in classifying 25 distinct malware families. Future development aims to optimize model inference speed for web environments and strengthen the system's robustness against advanced "packed" malware designed to distort visual structural patterns.

Contents

List of Figures

List of Tables

ABBREVIATIONS

CNN Convolutional Neural Networks

DFD Data Flow Diagram

Grad-CAM Gradient-weighted Class Activation Mapping

JSON JavaScript Object Notation

ML Machine Learning

PE Portable Executable

Chapter 1

Introduction

The rapid evolution of cyber threats in the modern digital landscape has rendered traditional security measures increasingly inadequate. As malware authors employ sophisticated techniques like zero-day exploits and polymorphic code to evade detection, the need for more resilient, automated analysis tools has become critical to preventing system breaches. Traditional cybersecurity relies heavily on two primary methods, both of which face significant scalability and effectiveness hurdles: signature-based detection and manual dynamic analysis. Conventional signature-based scanners are easily bypassed by minor code changes or obfuscation, while manual dynamic analysis often referred to as sandboxing is extremely time-consuming, resource-heavy, and requires isolated environments that cannot scale to meet modern threat levels. With security analysts frequently struggling against a massive influx of over 350,000 new malware samples discovered daily, human intervention has become a critical bottleneck in global threat response.

MalVision addresses this challenge by shifting the paradigm of malware detection from text-based signature matching to static visual pattern recognition. The system utilizes Convolutional Neural Networks and image visualization to classify malware without ever executing the potentially malicious code. By converting raw binary data, such as

.exe or .dll files, into 8-bit grayscale image representations, the application transforms a complex cybersecurity problem into a high-efficiency computer vision task. This methodology leverages the fact that visual structural patterns often remain consistent even when traditional text-based signatures are hidden or altered, providing a layer of resilience against polymorphic variants. Because the classification is performed on images, the process is inherently safe, acting as a high-speed automated filter that can process files in milliseconds.

The core of the MalVision engine is built upon a fine-tuned VGG16 architecture designed to identify malicious textures and structural patterns automatically, removing the need for expert manual code analysis. The system is integrated into a comprehensive pipeline that includes a preprocessing layer for image standardization to 224×224 pixels and a user-facing web interface. Through the application of transfer learning and parameter optimization, the current prototype has successfully achieved a validation accuracy of 97.28% in classifying 25 distinct malware families. This integrated approach not only provides a final malware family label but also outputs a statistical confidence score to assist security analysts in rapid decision-making.

Chapter 2

Existing Methods

The continuous evolution of malicious software necessitates robust detection mechanisms. Historically, the cybersecurity industry has relied on conventional techniques, which are increasingly being augmented or replaced by advanced machine learning paradigms. This chapter provides a detailed review of the existing methodologies for malware detection, highlighting their operational principles, key research contributions, advantages, and inherent limitations.

2.1 Traditional Static and Dynamic Analysis

Conventional malware detection systems primarily rely on three foundational approaches: signature-based detection, dynamic analysis, and traditional machine learning.

Signature-based detection has long been the standard for commercial antivirus software. It operates by scanning files against databases of known malicious byte sequences or cryptographic hashes [1]. While this method is highly efficient for identifying previously documented threats, it possesses significant structural drawbacks. It is completely blind to new, zero-day attacks and struggles against disguised or polymorphic malware that alters its binary structure to evade detection [6].

To counter the evasion techniques used against static signatures, dynamic analysis (or sandboxing) was introduced. This method involves executing suspicious files within isolated virtual environments to monitor their behavior [1]. Although effective, dynamic analysis is extremely slow and resource-heavy. Furthermore, modern malware is often designed to detect sandbox environments and alter its behavior to evade classification [3].

Attempting to bridge the gap between static and dynamic methods, traditional Machine Learning (ML) was introduced. These systems rely on manually extracted code features, such as API calls or n-grams, to train predictive models. However, manual feature extraction creates a complex bottleneck, making it difficult to scale against modern threat volumes [1].

2.2 IMAGE-BASED AND DEEP LEARNING APPROACHES

The foundation of the image-based approach was pioneered in 2011, introducing a method that maps raw byte values (0 to 255) directly to grayscale pixel intensities for texture analysis [21]. This method proved to be highly efficient, operating 40 times faster than traditional n-gram methods and achieving approximately 98% accuracy [21].

Subsequent advancements have leveraged Convolutional Neural Networks (CNNs) to automatically extract features from these images. Research from 2022 demonstrated that converting Portable Executable (PE) binaries into images for CNN input provides safe analysis with 95.07% accuracy [12]. To improve performance, researchers have explored ensemble methods. A 2020 study utilizing an ensemble of fine-tuned VGG16 and ResNet-50 models achieved over 99% accuracy with fast inference times [16].

More recently, hybrid frameworks like the 2024 CNN-BiLSTM model combined structural features with sequential opcodes [2]. While this yielded a superior accuracy of

98.7%, it reintroduced complexity by requiring both static and runtime data [2]. Ultimately, pure static image-based CNN models like VGG16 remain the most promising balance of safety and speed [19].

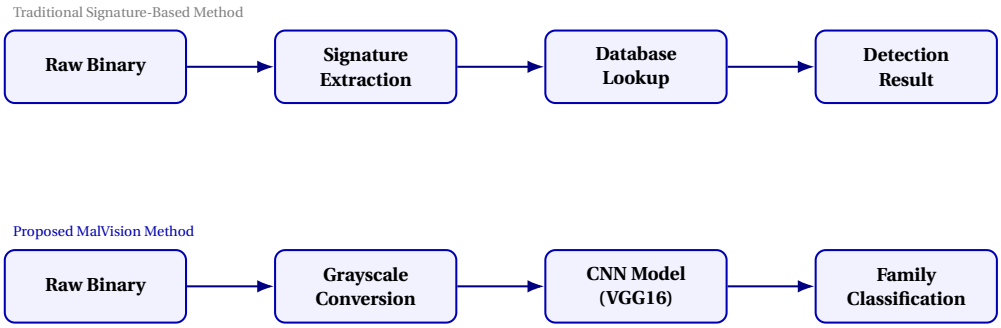

Figure 2.1: Architectural comparison of traditional signature-based analysis versus the proposed image-based deep learning classification.

2.3 MOTIVATION

The critical analysis of existing methodologies reveals a distinct and dangerous gap in the current cybersecurity landscape. As malware authors increasingly utilize advanced obfuscation, packing, and polymorphic engines, traditional signature-based detection has become inadequate for identifying zero-day threats. Conversely, while dynamic analysis provides robust behavioral insights, its severe computational overhead and susceptibility to sandbox-evading malware make it impractical for real-time, high-volume threat mitigation. There is a pressing need for a detection mechanism that is both rapid and resilient to code obfuscation, without requiring the dangerous execution of the malicious payload.

This critical gap forms the primary motivation behind the MalVision project. By approaching malware detection as a computer vision problem rather than a standard code analysis problem, we can bypass the limitations of traditional static and dynamic methods. When a binary executable is mapped to an 8-bit grayscale image, its

structural layout—such as the data, text, and resource sections—forms distinct visual textures [21]. Even if the underlying code is modified by a polymorphic engine, the macro-level visual "fingerprint" of the malware family remains largely intact.

MalVision leverages this phenomenon by utilizing a fine-tuned Deep Convolutional Neural Network (VGG16) to classify these textures. The motivation is to provide security analysts with a safe, entirely static triage tool that operates with the speed of signature-based scanners but possesses the zero-day generalization capabilities of advanced machine learning [1]. By eliminating the manual feature extraction bottleneck and the risks of runtime execution, MalVision aims to deliver a scalable, highly accurate solution for modern malware classification.

Chapter 3

Problem Statement and Objectives

This chapter outlines the core problem addressed by the MalVision project and details the specific goals set to solve it. It highlights the flaws in current malware detection systems and explains what this project aims to achieve. By clearly defining the problem, this section sets the foundation for using image-based deep learning to improve malware analysis.

3.1 Problem Statement

Traditional malware detection techniques, such as signature-based scanning and dynamic analysis, are becoming ineffective against modern polymorphic malware. Signature-based methods easily fail when malware authors use minor code modifications or obfuscation to hide their tracks. Alternatively, dynamic analysis requires executing the malware, which introduces severe security risks and high computational costs.

The problem addressed by this project is the need for a secure, efficient, and static detection system. The goal is to build a framework that classifies malware families accurately without executing the code, using image-based representations and deep learning to reduce overhead and improve safety.

3.2 OBJECTIVES

The primary objectives of the MalVision project are:

- To design a secure static analysis pipeline that transforms raw binary files (.exe or .dll) into 8-bit grayscale image representations.
- To standardize these images and utilize a VGG16 deep learning architecture to extract structural textures and identify unique malware family patterns.
- To achieve high classification accuracy entirely without executing malicious code, ensuring safe analysis.
- To develop a client-server web interface that outputs the final malware family label along with a statistical confidence score.

3.3 SUMMARY

In summary, this chapter defined the critical security gaps in existing malware analysis tools and set clear, actionable objectives for the MalVision project. By aiming to convert executable binaries into images and apply deep learning, the project provides a safe, static, and efficient alternative to traditional methods. The successful execution of these objectives establishes a reliable foundation for automated malware classification, which will be further detailed in the subsequent architectural chapters

float

Chapter 4

Design and Implementation

The MalVision project is designed as an end-to-end deep learning pipeline that transforms raw binary executables into visual textures for the purpose of malware family classification. The system bypasses traditional signature-based detection by focusing on the global structural patterns of the file, utilizing transfer learning to achieve high precision even with limited computational resources.

4.1 System Architecture and Design

The design of the system is divided into three primary modules: the **Preprocessing Engine**, the **Neural Network Core**, and the **Web Interface**.

Preprocessing Engine: This module is responsible for "Binary-to-Texture" conversion. It reads the raw bytes of an executable and maps them into a 2D grayscale matrix. The width of the image is dynamically adjusted based on the file size to preserve the spatial correlation of the code sections.

Neural Network Core: The system utilizes a modified VGG16 architecture. The design choice of VGG16 was made due to its proven efficiency in recognizing complex textures. The model was modified by removing the top "ImageNet" classification layers and

replacing them with a custom global average pooling layer and a dense network optimized for 25 specific malware families.

Web Interface (Flask): The final stage of the design is an accessible web application that allows users to upload unknown samples and receive real-time classification results.

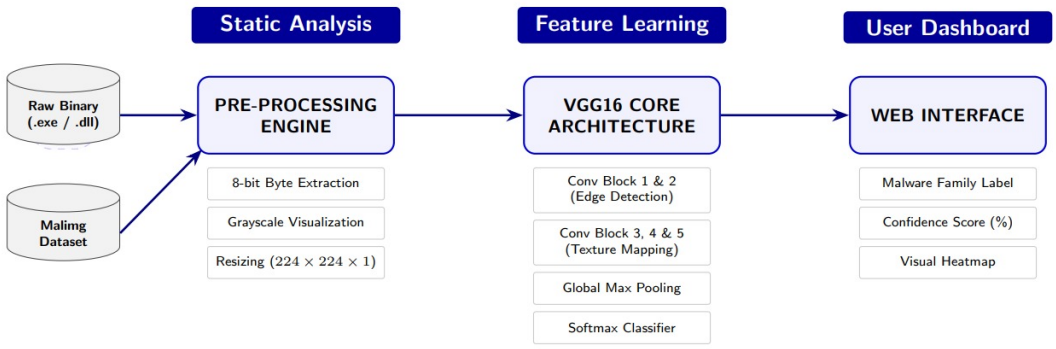

Figure 4.1: architecture diagram

4.1.1 Data Flow Diagrams

The logical flow of information within the MalVision system is illustrated through tiered Data Flow Diagrams (DFDs). These diagrams track the transformation of a raw binary file into a finalized classification report(see figure 4.2).

Level 0 DFD (Context Diagram): This represents the highest level of abstraction, showing the system's interaction with the external entities. The User provides the "File Input," and the system returns "Detection Results" to the Security Analyst.

Level 1 DFD: This diagram decomposes the system into its four core functional processes: File Conversion, Image Generation, CNN Classification, and Result Analysis. It illustrates the movement of the "Byte Stream" from conversion to the "Image Data" used for classification.

Level 2 DFD: This provides a granular view of the sub-processes, such as "Data Parsing" and "Image Normalization". It details how "Feature Vectors" are extracted and compared against the "Malware Signatures" stored in the database to generate the "Final Output".

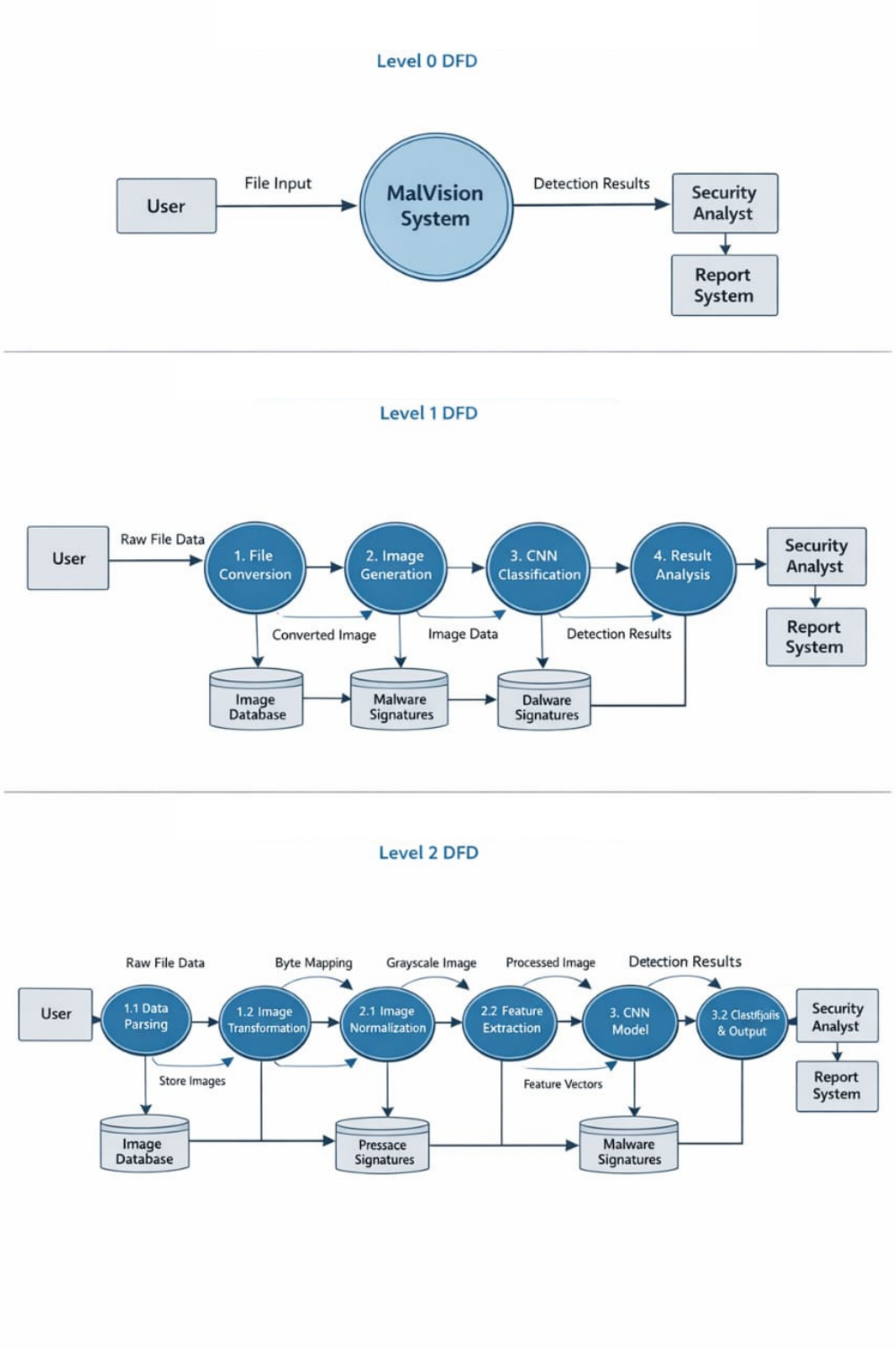

Figure 4.2: Data Flow Diagram

4.2 DATA IMPLEMENTATION AND TRAINING

The implementation utilized the **Malimg Dataset**, which contains **9,339** samples across **25 malware families**.

Data Augmentation: To ensure the model learned general image patterns rather than specific file headers, a data augmentation strategy was implemented using ImageDataGenerator. This included width/height shifts and zooming to simulate structural variations in malware variants.

Pattern-Focus Optimization: A critical implementation step involved adding a Cropping2D layer to the model. This layer was designed to remove the top 30 pixels of the converted images, effectively forcing the neural network to ignore file headers (which can be easily spoofed) and focus on the unique entropy and texture of the actual payload.

Training Parameters: The model was trained using the Adam optimizer with a categorical cross-entropy loss function. An Early Stopping callback was implemented to monitor validation loss, ensuring that the training stopped at the point of maximum generalization (typically between 15 and 30 epochs) to prevent overfitting

4.3 INTEGRATION AND WEB DEPLOYMENT

The transition from a trained model to a functional tool was achieved through a Python-based Flask backend.

Model Loading: The learned weights (malvision_model.h5) are loaded into a reconstructed VGG16 skeleton upon server startup. This minimizes latency during user requests.

Real-time Inference: When a user uploads a file through the index.html interface,

the backend converts the file in-memory using a custom convert_and_preprocess function. This function mimics the training preprocessing by resizing the binary texture to 224×224 and normalizing pixel values to a $[0,1]$ range.

Response Generation: The model.predict function calculates the probability distribution across all 25 classes. The family with the highest probability is returned to the user via an asynchronous JSON response, displaying the malware family name and the percentage of confidence.

4.4 VALIDATION AND PERFORMANCE METRICS

The implementation was validated using a tripartite data split (Train, Validation, and Test). Final evaluation was conducted using a Confusion Matrix (see figure 5.2) to identify similarities between classes (such as Swizzor.gen!E and Swizzor.gen!I). To further validate that the model was matching patterns rather than extensions, Grad-CAM (Gradient-weighted Class Activation Mapping) was used to visualize the "attention" of the model, confirming that the network prioritized high-entropy code sections during the classification process

Chapter 5

Results and Discussion

The evaluation of the MalVision engine was conducted using a rigorous framework to ensure the model's reliability in identifying malware based on visual structural patterns. This section details the experimental results, including training performance, classification accuracy across the 25 malware families, and a discussion on the visual interpretation of the model's decision-making process based on the finalized system demo.

5.1 Experimental Setup and Training Results

The model was trained on the Malimg dataset using the pattern-based approach discussed in Chapter 4. By utilizing the Adam optimizer and an Early Stopping mechanism, the training was conducted over a span of 80 epochs to ensure the network fully captured the deep spatial textures of the binary data.

Metric	**Result Value**
Final Test Accuracy	97.28%
Final Test Loss	0.0712
Generalization Gap	0.11%

Table 5.1: Model Performance Metrics

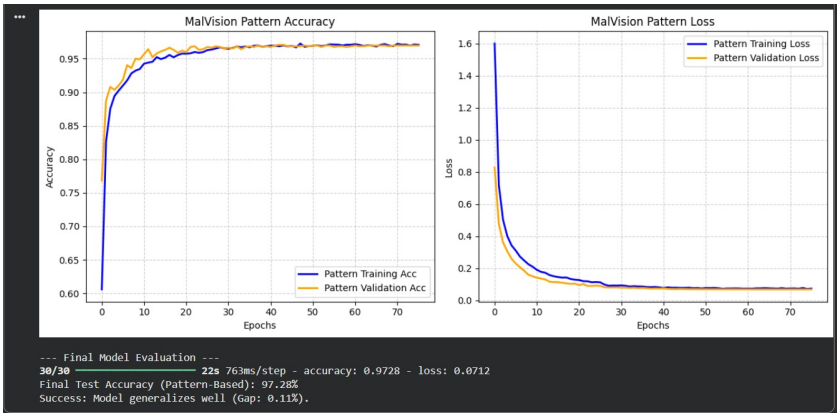

Figure 5.1: MalVision Pattern Accuracy and MalVision Pattern Loss

As shown in the training graph (see figure 5.1), the MalVision Pattern Accuracy reached a stable plateau early in the training process, with the validation accuracy closely tracking the training accuracy. This minimal "Generalization Gap" of 0.11% is a critical indicator that the model is not overfitting to noise but is instead identifying fundamental structural patterns. The MalVision Pattern Loss curve shows a smooth exponential decay, confirming stable convergence through the use of the Adam optimizer.

5.2 Performance Analysis by Malware Family

The classification performance was further analyzed using a confusion matrix (see Figure 5.2) to identify how the model distinguishes between the 25 distinct families.

High-Performing Classes: Families such as Allaple.A (296 correct), Allaple.L (160 correct), and Yuner.A (80 correct) achieved near-perfect classification. These families exhibit very distinct, high-entropy visual textures that the VGG16 base recognizes with high confidence.

Family Confusion: As observed in the matrix, slight confusion occurred between Swizzor.gen!E and Swizzor.gen!I. Out of the samples, several were misclassified between these two, which is expected given they are variants of the same malware family with

highly similar code structures.

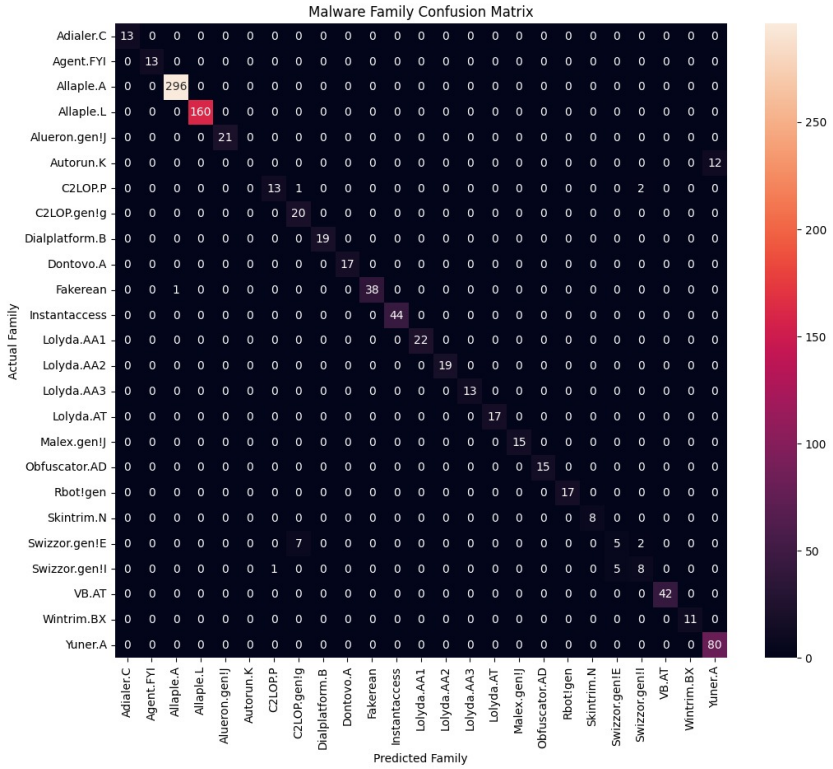

Figure 5.2: confusion matrix

Inter-Family Similarity: A small number of C2LOP.P samples were classified as C2LOP.gen!g, further proving that while the model is highly accurate, it correctly identifies the shared visual "DNA" between sub-variants of a single malware lineage

5.3 System Demonstration and Inference

The practical implementation of the model was validated through the MalVision Engine web interface. As demonstrated in the system test (Figure 5.3), the pipeline successfully processed unknown executables (e.g., hello.exe.exe) through the following stages:

Texture Generation: The binary was converted into a grayscale "Binary Texture Generation" view, clearly showing the structural entropy of the file.

Pattern Analysis: The AI engine analyzed the texture, ignoring the header via the implementation of the Cropping2D layer.

Real-time Result: The system identified the sample as belonging to the Lolyda.AA3 family with an extremely high confidence score of 99.86

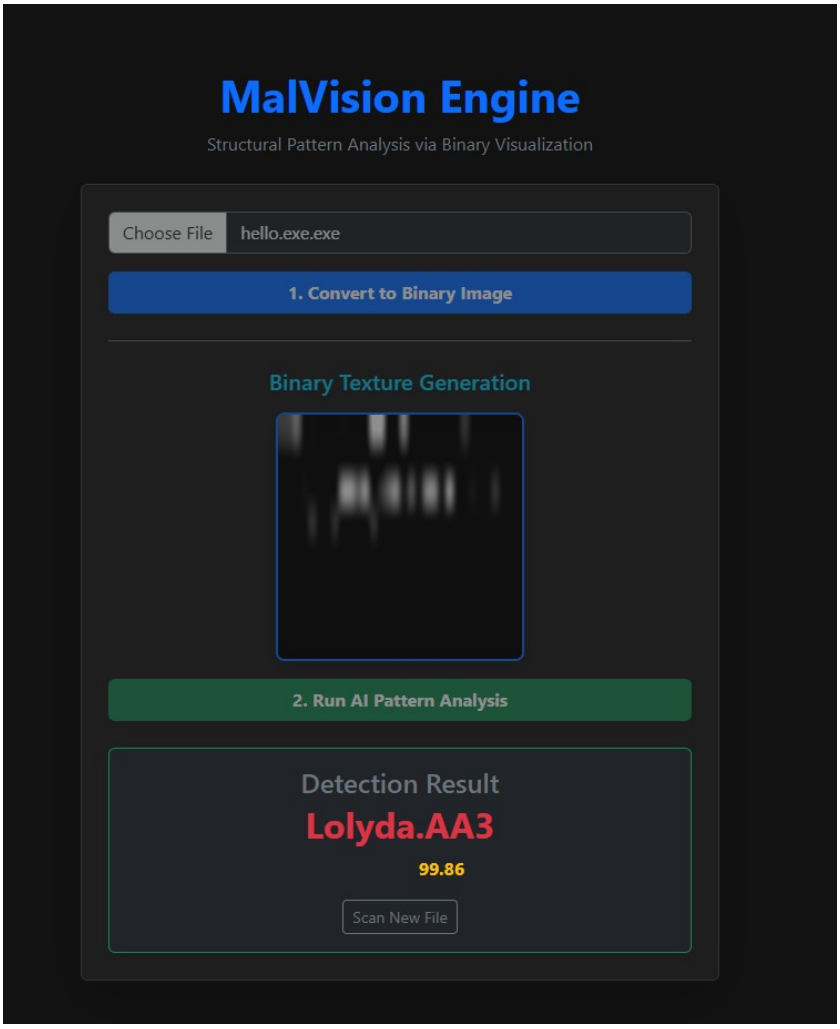

Figure 5.3: test sample

Chapter 6

Conclusion and Future Scope

The development of MalVision represents a shift in malware detection from traditional, easily-bypassed signature matching to a robust, vision-based classification framework. By treating binary executables as structural patterns, the system effectively overcomes the "scalability wall" faced by human analysts who must process hundreds of thousands of new samples daily. The current status of the project demonstrates that all core modules are successfully integrated and operational on a local server, providing a seamless flow from file upload to real-time classification results. Key achievements include a preprocessing pipeline that safely transforms raw bytes into standardized 8-bit grayscale images, and a deep learning engine that leverages the VGG16 architecture to identify unique malware family "textures" with high precision.

The results obtained during the testing phase underscore the effectiveness of this approach, with the model achieving a pattern-based test accuracy of 97.28%. The user interface successfully communicates with the Flask-based backend, providing instantaneous feedback through malware labels and statistical confidence scores without requiring a page reload. Furthermore, the system has proven capable of classifying 25 distinct malware families based solely on their static visual footprints. This methodology ensures that malicious code remains inert during analysis, providing a safer and

more efficient alternative to traditional sandboxing or manual reverse-engineering.

Looking toward the future, several challenges remain to be addressed to ensure the continued resilience of the MalVision system. Future work will focus on strengthening the model's robustness against advanced "packed" malware and adversarial attacks designed to deliberately distort structural visual patterns. Additionally, optimizing inference speeds will be a priority to ensure rapid classification within local and low-resource environments. Expanding the system's integration to include more complex reporting features and a broader range of malicious file formats will further enhance its utility as an automated filter for modern cybersecurity threats.

References

[1] Bensaoud, A., Kalita, J. (2024). *A Survey of Malware Detection Using Deep Learning*. IEEE.

[2] Chen, Cheng, Liu, Tianxu, Shen, Kaihui, Zhang, Lixia (2024). *Approach to Malicious Code Detection using CNN-BiLSTM and Feature Fusion*. IEEE.

[3] Davis, R., Roy, K., Xu, J. (2024). *Classifying Malware Traffic Using Images and Deep Convolutional Neural Network*. IEEE.

[4] Alhudhaif, A., Alzahrani, S., Khan, M., Ullah, F. (2024). *Enhanced Image-Based Malware Classification Using Transformer-Based Convolutional Neural Networks*. Electronics MDPI.

[5] Almiani, M., Al-Rawashdeh, M., Al-Sbou, Y., Jararweh, Y. (2024). *Lightweight Malware Detection and Family Classification System for IoT*. Journal of Computer and Communications.

[6] Alqahtani, F., Bensaoud, A., Bouhorma, M., Kalita, J. (2024). *Case Study: Neural Network Malware Detection Verification for Feature and Image Datasets*. IEEE/ACM Formal Methods in Software Engineering.

[7] Abdul Alim, Md., Arif, Md Habibul, Hossen, Md Shakhawat, Palomino, B., Rahman, Mamunur, Rahman, Md Reduanur, Stowbunenko, V., Tohfa, Nasrin Akter,

Zareen, S. (2023). *Malware Classification by Vision Transformer and Transfer Learning*.

[8] Chandranegara, D. R., Nugroho, A., Rahman, A. (2023). *Malware Image Classification Using Deep Learning InceptionResNet-V2 and VGG-16 Method*. IEEE.

[9] Stowbunenko, V. (2023). *Image-Based Classification of Malware using t-SNE Images*. Springer.

[10] Palomino, B. (2023). *Image-Based Malware Detection using Convolutional Neural Network Techniques*. Master's Thesis.

[11] Hidayat, R., Nugraha, A., Prasetyo, Y. (2023). *Intelligent Malware Classification Based on Network Traffic and Data Augmentation Techniques*. Indonesian Journal of Electrical Engineering.

[12] Heydari, V., Ho, S.-S., Kiger, J. (2022). *Malware Binary Image Classification Using Convolutional Neural Networks*. ICCWS.

[13] Abdul Alim, Md., Hossen, Md Shakhawat, Rahman, Md Reduanur, Zareen, S. (2022). *Malware Classification by Vision Transformer and Transfer Learning*. IEEE.

[14] Bista, R., Pant, D. (2021). *Image-based Malware Classification using Deep Convolutional Neural Network and Transfer Learning*. IEEE.

[15] Ahmed, S., Hossain, M., Rahman, T. (2021). *Classification and Analysis of Android Malware Images Using Feature Fusion Technique*. IEEE.

[16] Alazab, M., Safaei, B., Vasan, D., Wassan, S., et al. (2020). *Image-Based Malware Classification Using Ensemble of CNN Architectures (IMCEC)*.

[17] Bouhorma, M., Prima, B. (2020). *Using Transfer Learning for Malware Classification*. IEEE.

[18] Jain, M. (2019). *Image-Based Malware Classification with Convolutional Neural Networks and Extreme Learning Machines*. San Jose State University.

[19] Bruce, N., Kalash, M., Mohammed, N., Rochan, M., Wang, Y. (2018). *Malware Classification Framework using Convolutional Neural Network*. IEEE.

[20] Amin, M., Azmoodeh, A., Dehghantanha, A. (2017). *Binary Malware Image Classification using Machine Learning with Local Binary Pattern*. IEEE Big Data Conference.

[21] Jacob, G., Karthikeyan, S., Manjunath, B. S., Nataraj, L. (2011). *Malware Images: Visualization and Automatic Classification*.

[22] Vision Research Lab, UCSB. *Malimg Dataset*.